```
when(src) { D print(src.balance) }
```

```
when(dst) { B dst.balance += 10; }
```

```
when(src) { A src.balance += 10; }
```

The semantics have incidental restrictions on executions

33

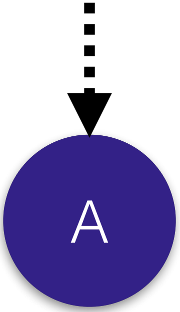
But, the semantics and all it’s implementation prohibit okay behaviour orders. ⚠️ src

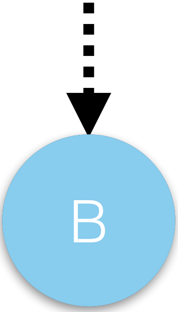
dst

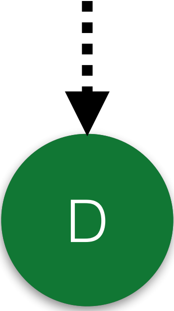
```
when(dst) { C print(dst.balance) }
```

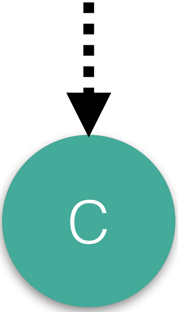
Update path (`src` then `dst`)

Diagnostic path (`dst` then `src`)


A


B


D


C

The semantics tie the execution order to the spawning order.

When executed concurrently, the update path and the diagnostic path are seemingly independent.

The semantics have incidental restrictions on executions

Update path (`src` then `dst`)

```
when(src) { A src.balance += 10; }
when(dst) { B dst.balance += 10; }
```

Diagnostic path (`dst` then `src`)

```
when(dst) { C print(dst.balance) }
when(src) { D print(src.balance) }
```

When executed concurrently, the update path and the diagnostic path are seemingly independent.

But, the semantics and all it's implementation prohibit okay behaviour orders. ⚠️

The semantics tie the execution order to the spawning order.

`src` `dst`

The semantics have incidental restrictions on executions

Update path (`src` then `dst`)

```
when(src) { A src.balance += 10; }
when(dst) { B dst.balance += 10; }
```

Diagnostic path (`dst` then `src`)

```
when(dst) { C print(dst.balance) }
when(src) { D print(src.balance) }
```

Order

Permitted